

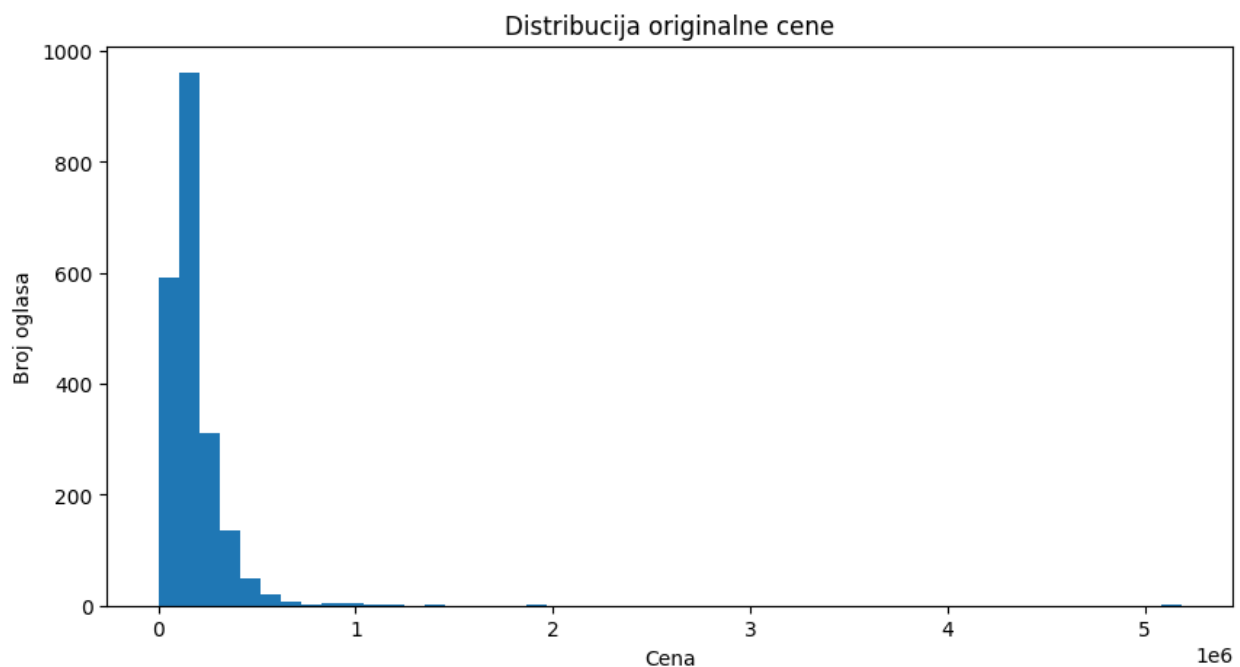
REPORT – LINEAR REGRESSION

1. Exploratory Data Analysis (EDA)

I started working with the data by loading the training set and performing a basic overview of the data structure. Through the analysis, I determined the number of instances, column types, and the presence of potential problems in the data, such as missing and improperly formatted values.

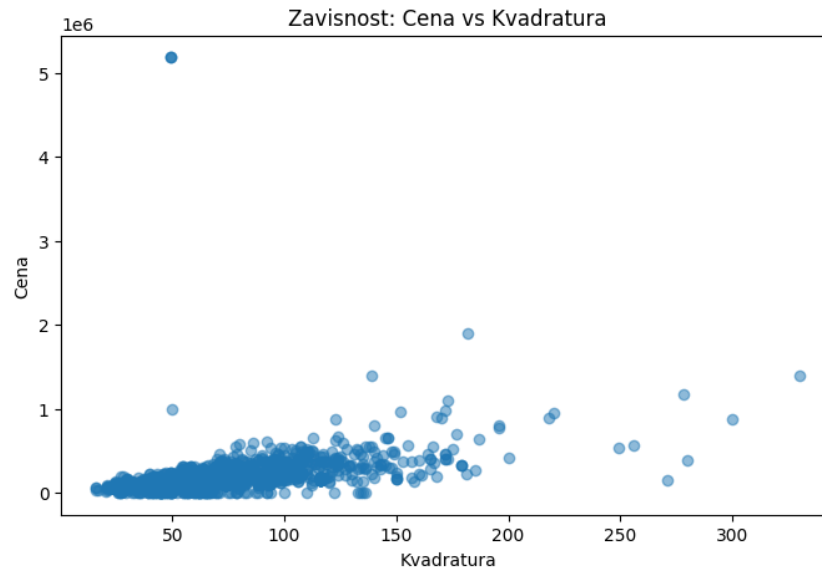
The first step was checking for missing values. Although standard functions did not immediately indicate a problem, through a more detailed analysis, I noticed that numerical columns contained string values masking actual missing data. Because of this, I converted all numerical columns using `pd.to_numeric(errors='coerce')`, whereby irregular values were converted to NaN, enabling their further processing.

Then, I analyzed the distribution of the target variable – the real estate price.

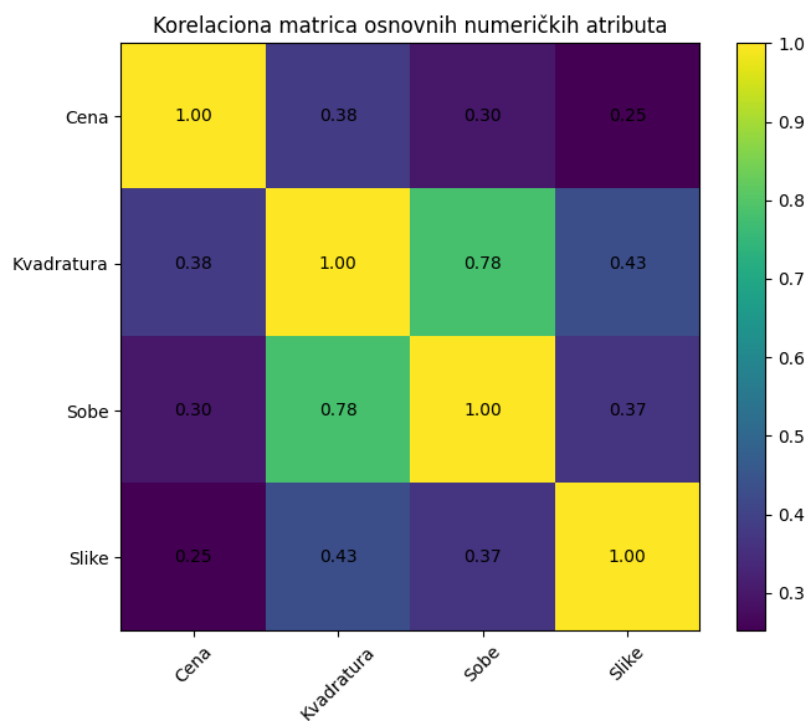


The analysis showed that the distribution is highly right-skewed, with a large number of extreme values.

In addition, the relationship between price and square footage was analyzed.



The expected linear dependence was not clearly pronounced, indicating that square footage alone is not sufficient for an accurate price prediction.

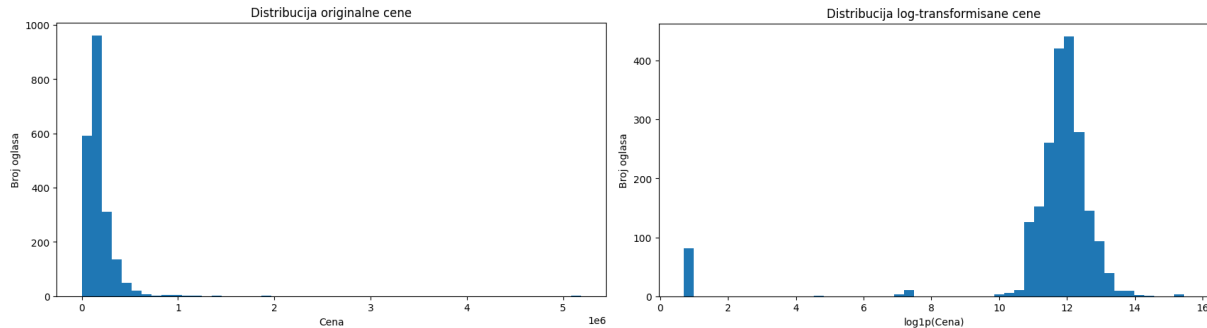


The conclusion of the EDA phase is that:

- the data contains outliers
- the distribution of the target variable is not suitable for direct modeling
- additional feature engineering is needed

2. Target Variable Transformation

Due to the pronounced asymmetry of the price distribution, a logarithmic transformation ($\log_{10}p$).



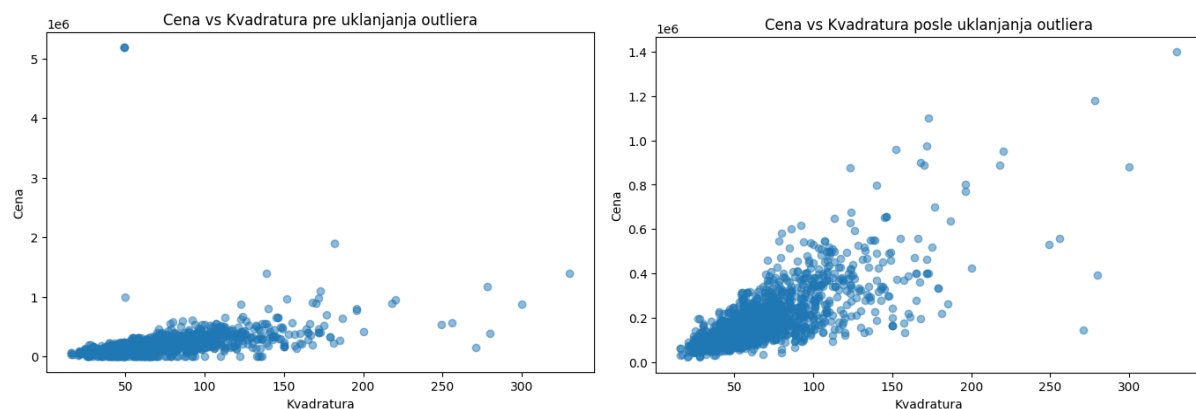
The logarithmic transformation did not completely create an ideal normal distribution, but it significantly reduced the impact of extremely high prices and gathered values into a more compact range. This made the target variable more suitable for linear models, as the model no longer attempts to learn directly from a very wide range of prices.

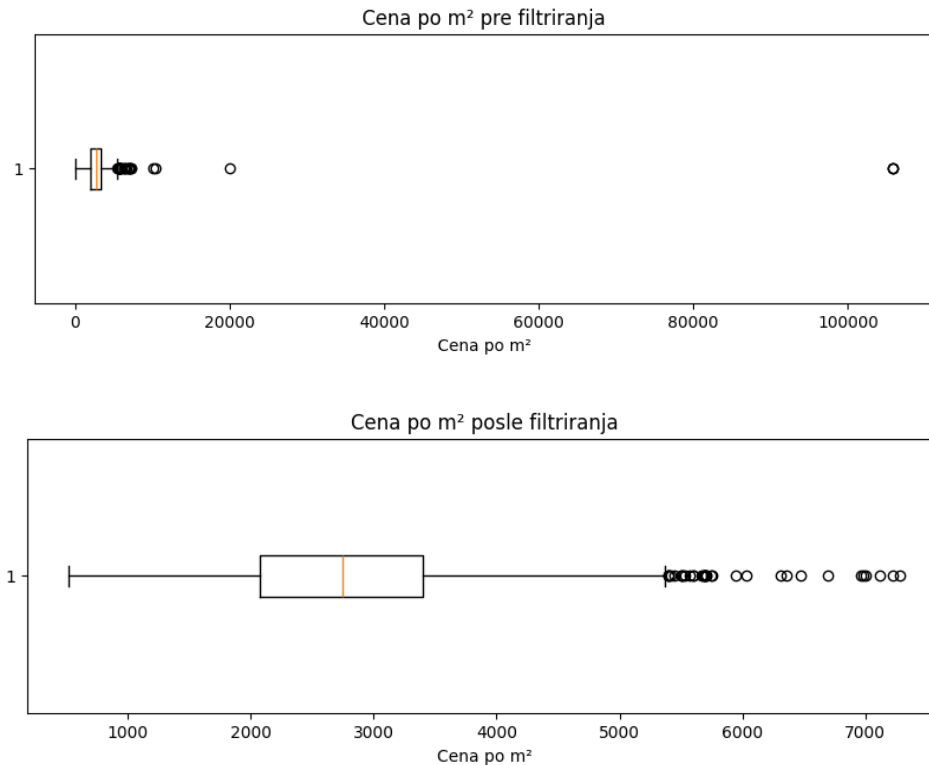
3. Outlier Removal

Extreme data points that can significantly impact the model were identified.

The following criteria were used:

- price > 15,000
- price per square meter between 500 and 8000
- Z-score < 3 (on log-transformed price)





This procedure removed extreme cases that would negatively affect the model training.

4. Data Preprocessing and Feature Engineering

4.1 Numerical Values Processing

Numerical columns were converted into the appropriate format, and missing values were filled with the median.

	kolona	NaN pre konverzije	NaN posle to_numeric
0	Cena	0	0
1	Kvadratura	0	0
2	Sobe	0	0
3	Slike	0	5

The table shows that after the numerical conversion, additional NaN values appeared in the column *Images*. This means that there were values in the original set that were not correctly written numerically. Only after this conversion was median imputation applied.

The median was chosen because it is robust to outliers.

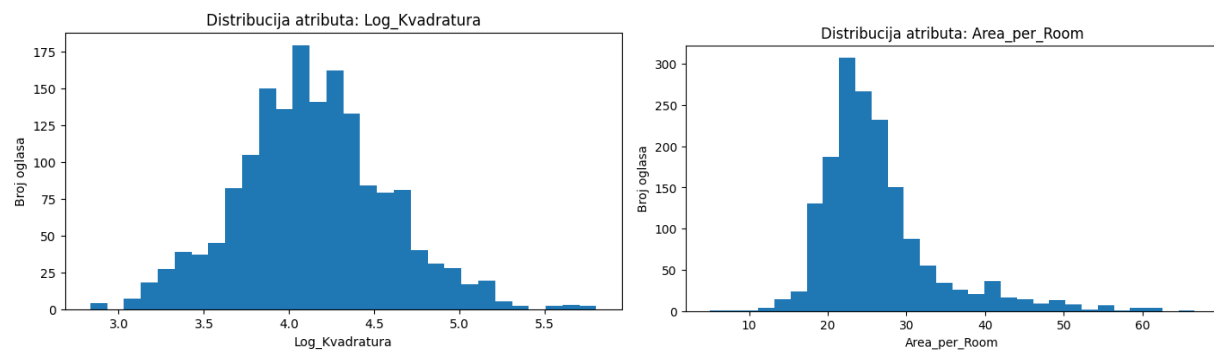
Learned medians from the training set:

```
{'images': np.float64(11.0), 'area': np.float64(60.5), 'rooms': np.float64(2.5),  
'floor': np.float64(2.0)}
```

4.2 Creating New Features

To improve the model's predictive power, additional features were created:

- Log_Kvadratura
- Area_per_Room
- Ima_Parking_Ili_Garazu
- Je_Prizemlje
- Je_Suteren
- Je_Investitor



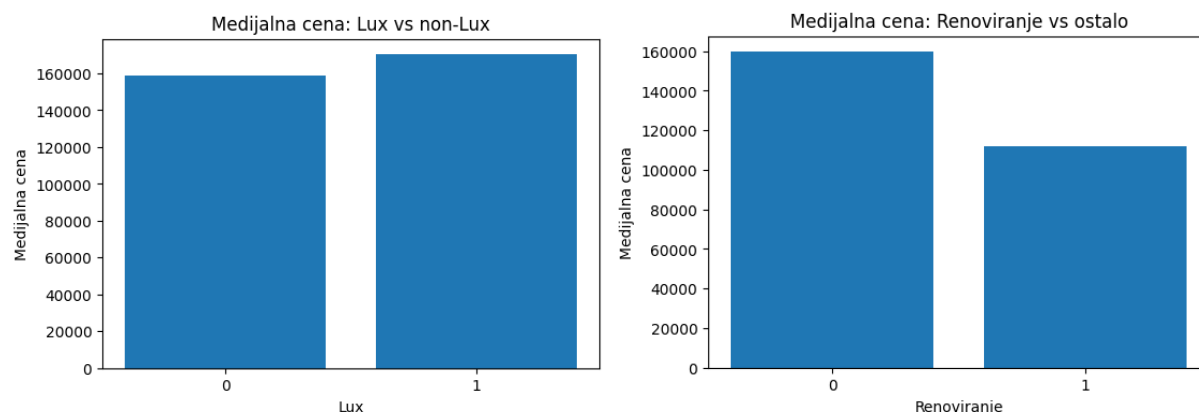
korelacija_sa_cenom	
Cena	1.000000
Log_Kvadratura	0.699599
Area_per_Room	0.212649
Sprat_Num	0.186470
Ima_Parking_Ili_Garazu	0.176355
Lux	0.088660
Renoviranje	-0.033892
Je_Investitor	-0.062859
Je_Suteren	-0.073743
Je_Prizemlje	-0.105475

Although not all new features have a strong individual correlation with the price, they add useful context to the model. Particularly important is *Log_Kvadratura*, while the other binary features help the model better explain the price through a combination of multiple signals.

4.3 Information Extraction from Text

The "Naziv" (Name) column contains useful information about the real estate. The following features were extracted from it:

- Lux
- Renovation

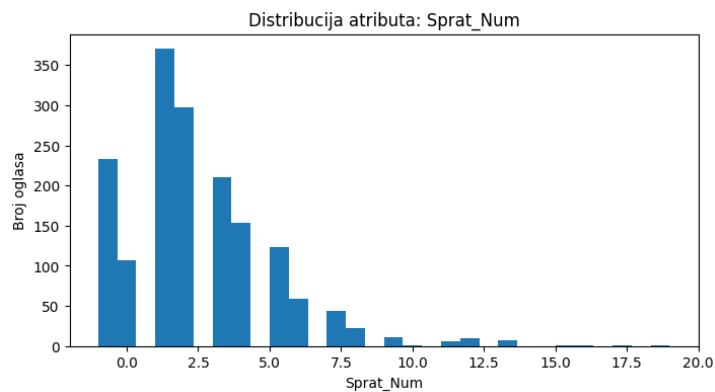


The graph shows that listings marked as *Lux* have a slightly higher median price, while listings containing terms related to renovation have a lower median price. This is logical, because terms like "investment," "renovation," or "adaptation" often point to an apartment that requires additional investments.

With this approach, the model gains additional context that is not directly available in numerical form.

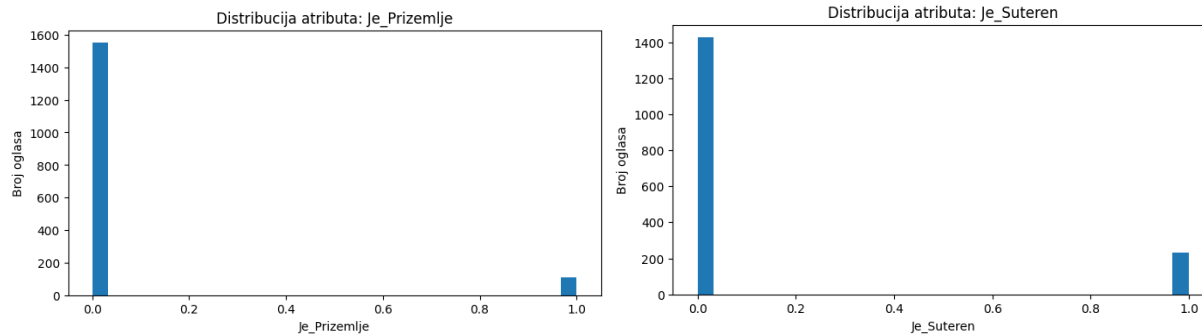
4.4 Floor Processing

The "Sprat" (Floor) column was standardized by mapping textual values to numerical ones.



Binary features were also added:

- Je_Prizemlje
- Je_Suteren



5. Categorical Features Encoding

5.1 One-Hot Encoding

One-Hot Encoding was applied for the "Grad" (City) feature.

Number of unique cities: 5

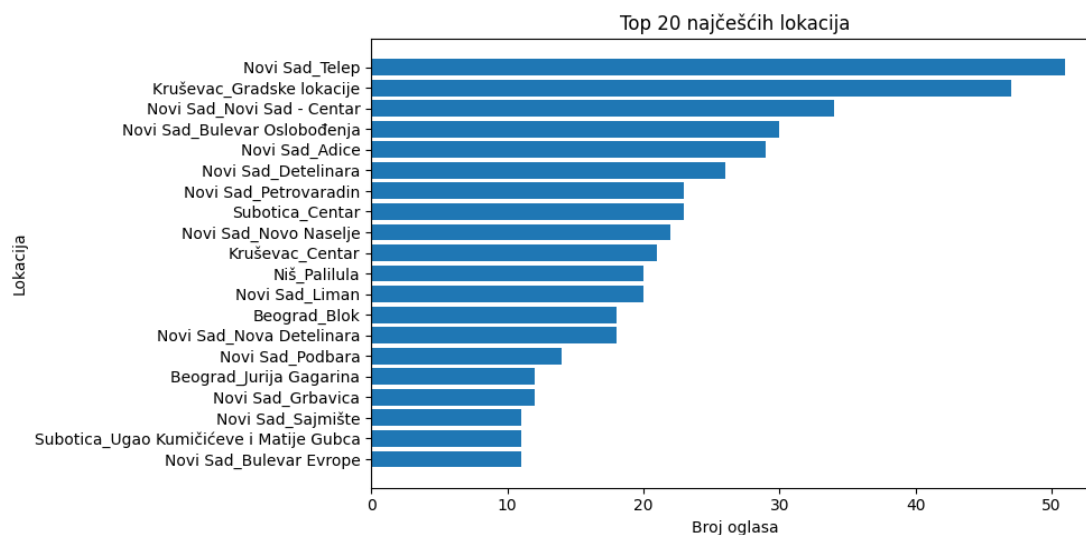
Number of unique merged locations: 709

5.2 Target Encoding

Target Encoding was used for the location feature.

Target Encoding allows a categorical variable to be represented by a numerical value based on the target (price), without increasing dimensionality.

It is important to note that this approach does not introduce non-linearity into the model, but only transforms the input data.



	broj_oglasa	medijalna_cena	medijalna_cena_m2
Lokacija_Spojeno			
Novi Sad_Telep	51	129000.0	2700.000000
Kruševac_Gradske lokacije	47	98000.0	1492.307692
Novi Sad_Novi Sad - Centar	34	176375.0	3294.047619
Novi Sad_Bulevar Oslobođenja	30	166000.0	2886.450268
Novi Sad_Adice	29	116600.0	2000.000000
Novi Sad_Detelinara	26	129908.0	2953.105263
Novi Sad_Petrovaradin	23	153070.0	2223.863636
Subotica_Centar	23	125000.0	1609.523810
Novi Sad_Novo Naselje	22	129375.0	2577.718703
Kruševac_Centar	21	121900.0	1739.130435
Niš_Palilula	20	107100.0	1839.195455
Novi Sad_Liman	20	188700.0	3059.507042
Beograd_Blok	18	254500.0	3808.979167
Novi Sad_Nova Detelinara	18	146070.0	2986.811594
Novi Sad_Podbara	14	130920.0	2727.500000
Novi Sad_Grbavica	12	163035.0	2948.979592
Beograd_Jurija Gagarina	12	247500.0	3859.558824
Novi Sad_Sajmište	11	159650.0	2956.481481
Subotica_Ugao Kumičićeve i Matije Gubca	11	86064.0	1623.849057
Novi Sad_Veternik	11	128750.0	1650.970874

6. Data Splitting and Validation

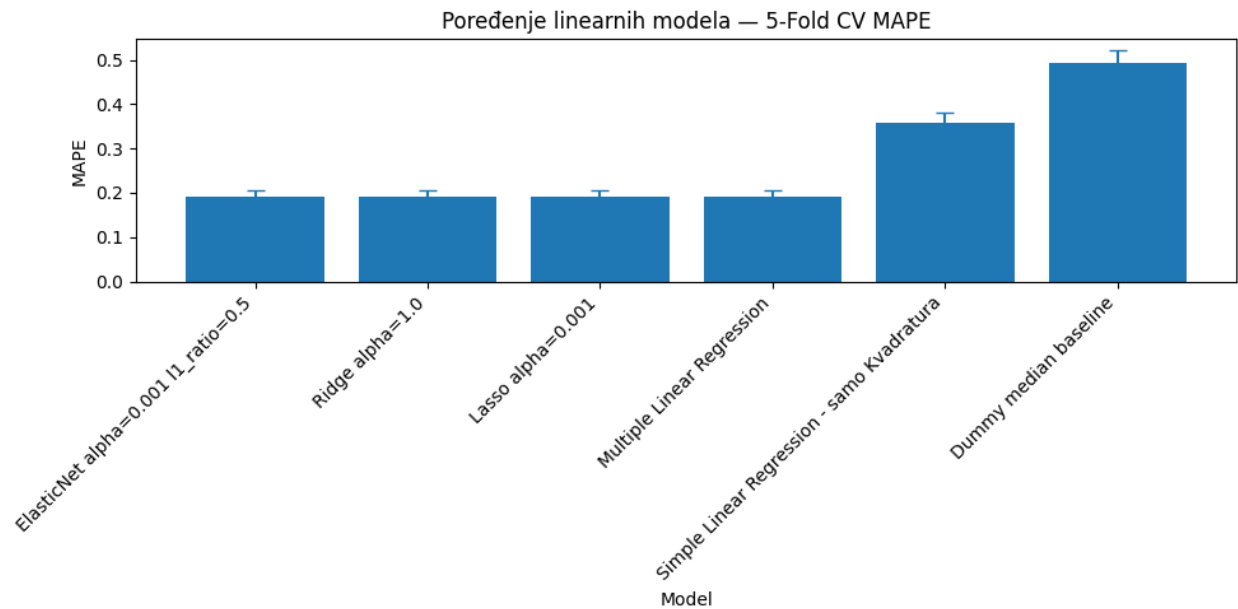
In order to avoid data leakage (*data leakage*), all transformations were learned exclusively on the training set.

A K-Fold cross-validation method with MAPE metric was used for model evaluation.

7. Comparison of Linear Models

The following models were tested:

- Linear Regression
- Ridge
- Lasso
- ElasticNet

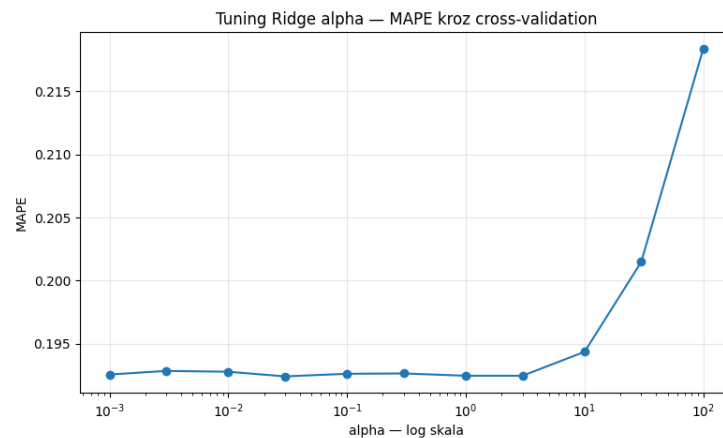


The cross-validation evaluation results show that models using all prepared features achieve a significantly lower MAPE error than simple linear regression relying solely on square footage, as well as the Dummy baseline model. Ridge, Lasso, and ElasticNet yield very similar results, confirming that regularization helps stabilize the linear model. Ridge was chosen as the final solution due to a good balance between simplicity, stability, and interpretability.

8. Ridge Model Tuning

To find the optimal value of the hyperparameter α (alpha), an analysis was conducted through K-Fold cross-validation using MAPE as the metric.

Ridge regression introduces L2 regularization, which penalizes large model coefficients. The α parameter directly controls the strength of this penalization. Therefore, a wide range of α values was tested on a logarithmic scale.



By analyzing the graph, the following can be observed:

- for very small α values, the model lacks sufficient regularization
- as α increases, there is a slight improvement in performance
- after a certain point, a further increase in α leads to a deterioration in results

The optimal α value is found in the region where MAPE reaches a minimum, representing the best balance between bias and variance.

Based on this analysis, the value $\alpha = 1.0$ was selected because:

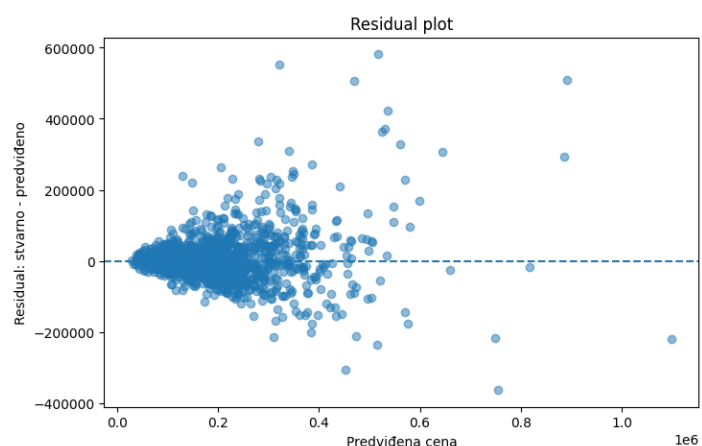
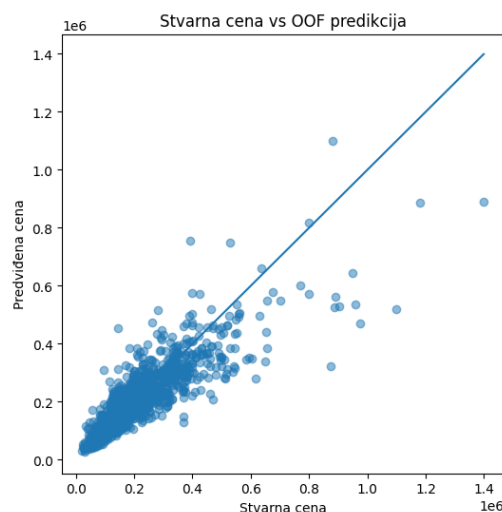
- provides stable results through cross-validation
- prevents overly large coefficients
- does not introduce excessive restriction to the model

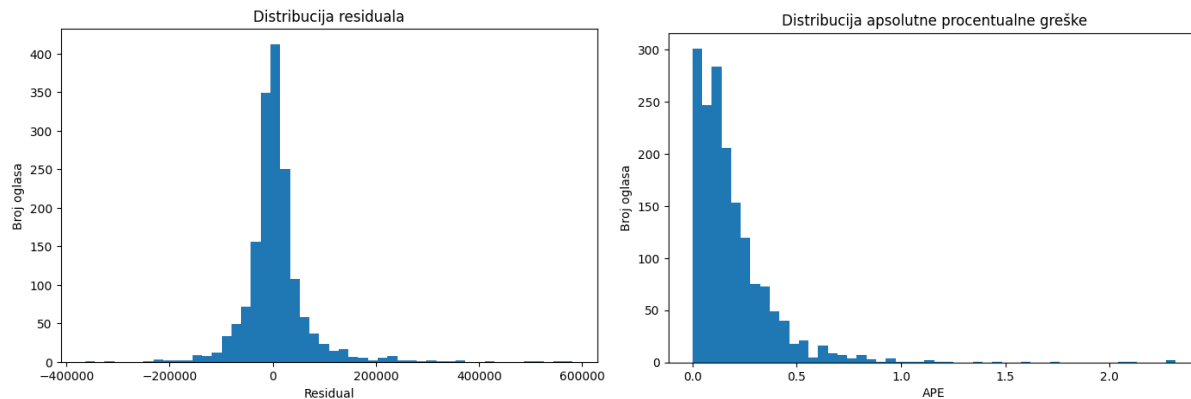
This approach ensures that the model is flexible enough to learn patterns from the data, yet sufficiently regularized to generalize to unseen examples.

9. Final Model

The final solution represents a pipeline that includes:

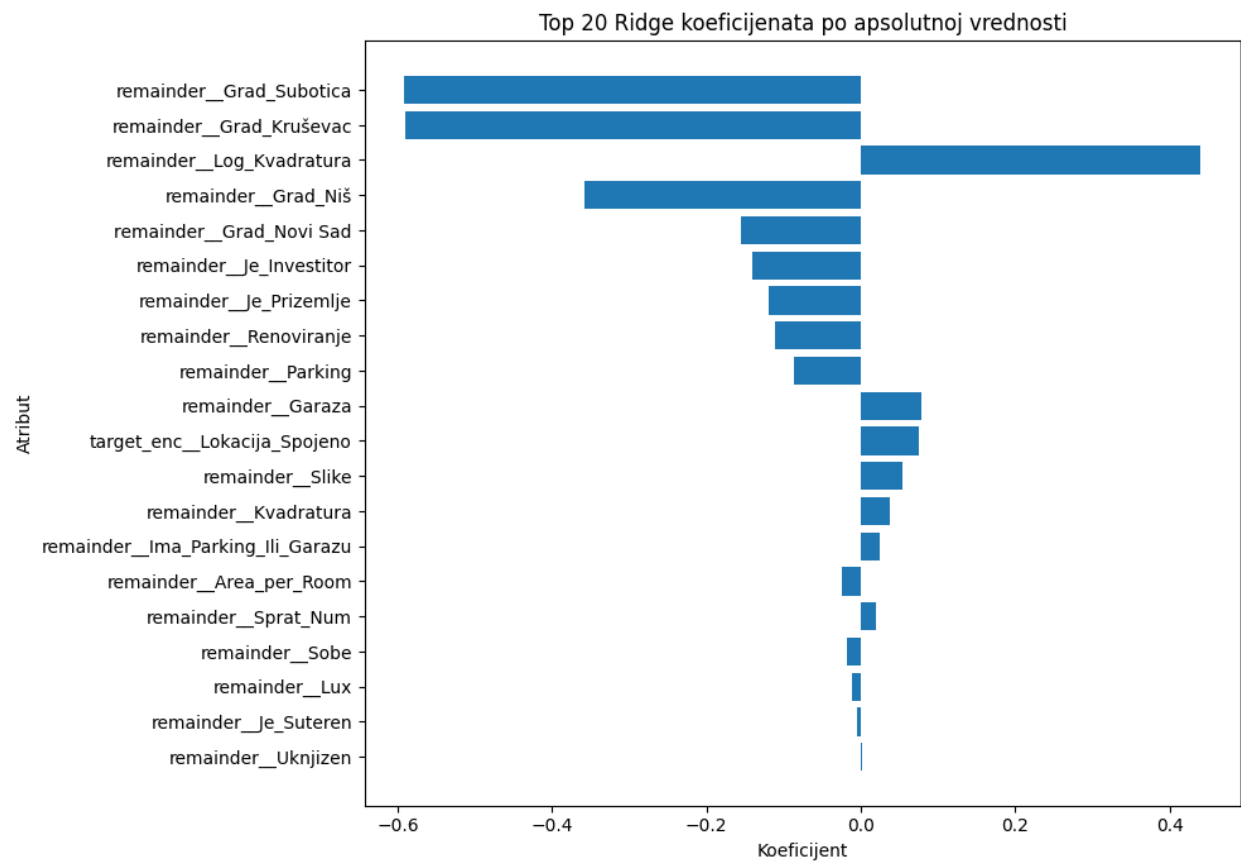
- Target Encoding
- RobustScaler
- Ridge regression
- log transformation of the target variable





The model shows good stability on most examples, which is seen through the concentration of residuals around zero and a large number of small percentage errors. However, there are larger deviations for more expensive real estates, which is expected as such examples are rarer and harder to predict accurately.

10. Model Interpretation



By analyzing the Ridge coefficients, one can roughly estimate which features most strongly affect the model's prediction. Since the features were previously scaled and transformed through the pipeline, the coefficients are interpreted relatively: a positive coefficient increases the prediction, while a negative one decreases it.

11. Conclusion

Through detailed data analysis, careful preprocessing, and systematic testing of the models, a stable linear model for predicting real estate prices was developed.

Key success factors were:

- log transformation of the target variable
- outlier removal
- feature engineering
- Target Encoding
- regularization (Ridge)

The model achieves a competitive and stable MAPE error and represents the selected final solution among the tested linear models.

12. Use of LLM Tools

During the development of this project, Artificial Intelligence (LLM) tools were used exclusively as assistants to speed up coding and solve specific syntax problems, while the solution architecture, feature engineering, and decision-making were designed independently based on exploratory analysis. Specifically, AI was used for:

- **Regular expressions (Regex) syntax:** For generating precise patterns when parsing the text column "Naziv" (to efficiently extract all variations of words like lux, renovated, basement).
- **Pandas manipulations optimization:** Help with refactoring code to avoid frequent SettingWithCopyWarning warnings when creating derived features.
- **Visualization formatting (Matplotlib/Seaborn):** For generating "boilerplate" code when plotting graphs (adjusting gap size, colors, and axis rotation) to obtain clear visualizations faster.

13. References

1. Course materials and lecture notes from the Faculty of Technical Sciences (for the theoretical basis of JL, VL, L1/L2 regularization, KNN algorithm...)
2. Towards Data Science – Regularization in Machine Learning (Available at: <https://towardsdatascience.com/regularization-in-machine-learning-6fbc4417b1e5/>)

3. Official Scikit-learn documentation (for model implementation and understanding hyperparameters). (Available at: https://scikit-learn.org/stable/supervised_learning.html)
4. GeeksforGeeks: House Price Prediction using Machine Learning in Python (Available at: <https://www.geeksforgeeks.org/machine-learning/house-price-prediction-using-machi%20ne-learning-in-python>)
5. Towards Data Science: Encoding Categorical Data Explained: A Visual Guide with Code Example for Beginners (Available at: <https://towardsdatascience.com/encoding-categorical-data-explained-a-visual-guidewith-code-example-for-beginners-b169ac4193ae/>)